

Spring архитектура

Inversion of control

Центральной частью Spring является подход Inversion of Control, который позволяет конфигурировать и управлять объектами Java с помощью рефлексии. Вместо ручного внедрения зависимостей, фреймворк забирает ответственность за это посредством контейнера. Контейнер отвечает за управление жизненным циклом объекта: создание объектов, вызов методов инициализации и конфигурирование объектов путём связывания их между собой.

Объекты, создаваемые контейнером, также называются управляемыми объектами (beans). Обычно, конфигурирование контейнера, осуществляется путём внедрения аннотаций (начиная с 5 версии J2SE), но также, есть возможность, по старинке, загрузить XML-файлы, содержащие определение bean'ов и предоставляющие информацию, необходимую для создания bean'ов.

Плюсы такого подхода:

- отделение выполнения задачи от ее реализации;
- легкое переключение между различными реализациями;
- большая модульность программы;
- более легкое тестирование программы путем изоляции компонента или проверки его зависимостей и обеспечения взаимодействия компонентов через контракты.

Dependency Injection(DI)

Под DI понимают то Dependency Inversion (инверсию зависимостей, то есть попытки не делать жестких связей между вашими модулями/классами, где один класс напрямую завязан на другой), то Dependency Injection (внедрение зависимостей, это когда объекты котиков создаете не вы в main-е и потом передаете их в свои методы, а за вас их создает спринг, а вы ему просто говорите что-то типа "хочу сюда получить котика" и он вам его передает в ваш метод). Мы чаще будем сталкиваться в дальнейших статьях со вторым.

Внедрение зависимости (Dependency injection, DI) — процесс, когда один объект реализует свой функционал через другой. Является специфичной формой «инверсии управления» (Inversion of control, IoC), когда она применяется к управлению зависимостями. В полном соответствии с принципом единой обязанности объект отдаёт заботу о построении требуемых ему зависимостей внешнему, специально предназначенному для этого общему механизму.

Связывание и @Autowired

Процесс внедрения зависимостей в бины при инициализации называется Spring Bean Wiring. Считается хорошей практикой задавать явные связи между зависимостями, но в Spring предусмотрен дополнительный механизм связывания @Autowired. Аннотация может использоваться над конструктор, поле, сеттер-метод или метод конфигурации для связывания по типу. Если в контейнере не будет обнаружен необходимый для вставки бин, то будет выброшено исключение, либо можно указать @Autowired(required = false), означающее, что внедрение зависимости в данном месте не обязательно.

Типы связывания:

- autowire byName,
- autowire byType,
- autowire by constructor,
- autowiring by @Autowired and @Qualifier annotations

Начиная со Spring Framework 4.3, аннотация @Autowired для конструктора больше не требуется, если целевой компонент определяет только один конструктор. Однако, если доступно несколько конструкторов и нет основного/стандартного конструктора, по крайней мере один из конструкторов должен быть аннотирован @Autowired, чтобы указать контейнеру, какой из них использовать.

Для чего нужен Spring

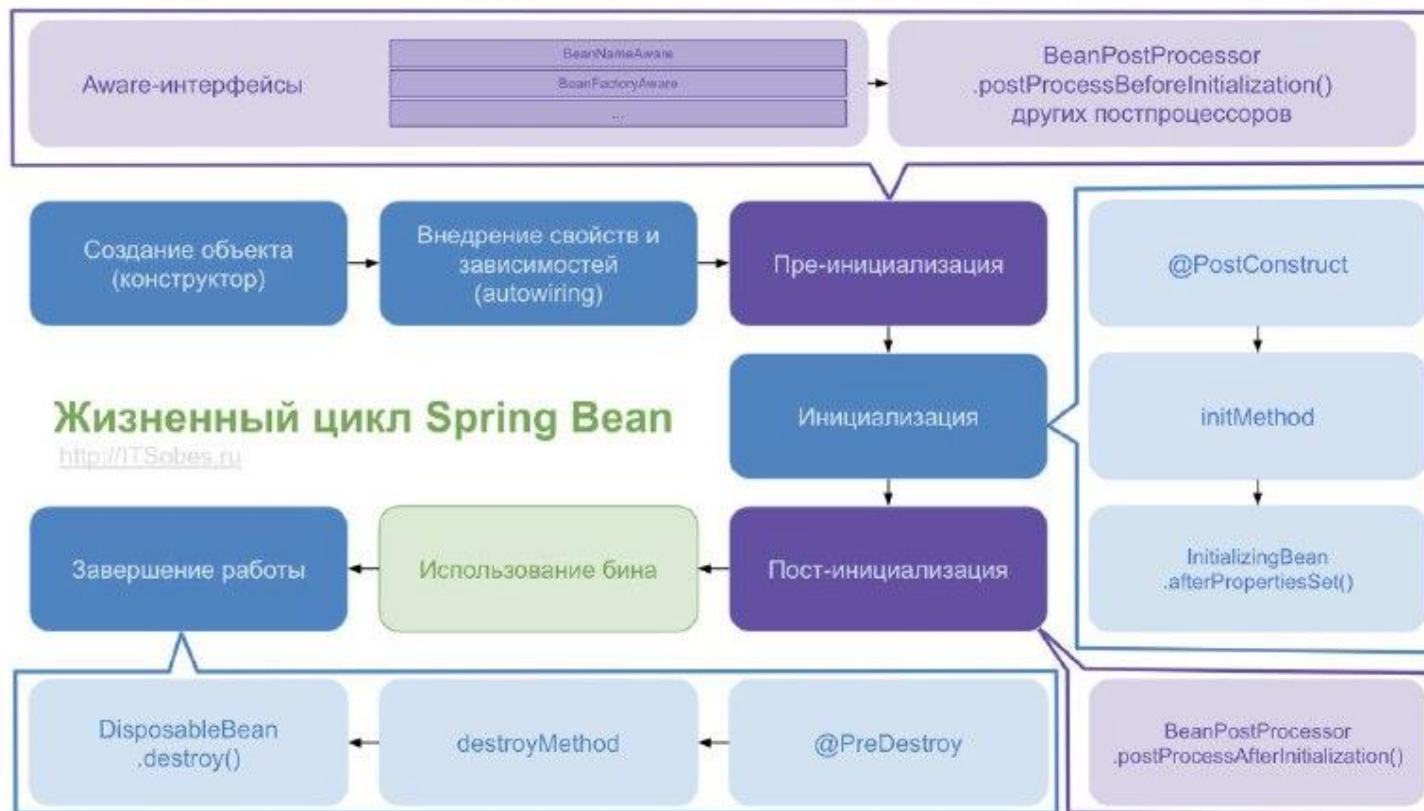
- Для более быстрого и легкого создания приложений — набор инструментов фреймворка позволяет выполнять те же задачи с меньшим количеством затрат, чем при написании с нуля.
- Для архитектурной «гибкости»: Spring универсален, поэтому позволяет реализовать нестандартные решения.
- Для гибкого использования возможностей — к проекту можно подключать разнообразные модули и тем самым настраивать инструментарий под свои нужды.
- Для удобного построения зависимостей, благодаря которому разработчики могут сконцентрироваться на логике приложения, а не на том, как подключить одно к другому.
- Для реализации парадигмы аспектно-ориентированного программирования, о котором мы подробнее расскажем ниже.
- Для решения задач, связанных со связями между компонентами или разными приложениями, для доступа различных частей системы друг к другу и многого другого.

Этапы поднятия context



- Контейнер создается при запуске приложения
- Контейнер считывает конфигурационные данные (парсинг XML, JavaConfig)
- Из конфигурационных данных создается описание бинов (BeanDefinition) BeanDefinitionReader
- BeanFactoryPostProcessors обрабатывают описание бина
- Контейнер создает бины используя их описание
- Бины инициализируются — значения свойств и зависимости внедряются в бин (настраиваются)
- BeanPostProcessor запускают методы обратного вызова(callback methods)
- Приложение запущено и работает
- Инициализируется закрытие приложения
- Контейнер закрывается
- Вызываются callback methods

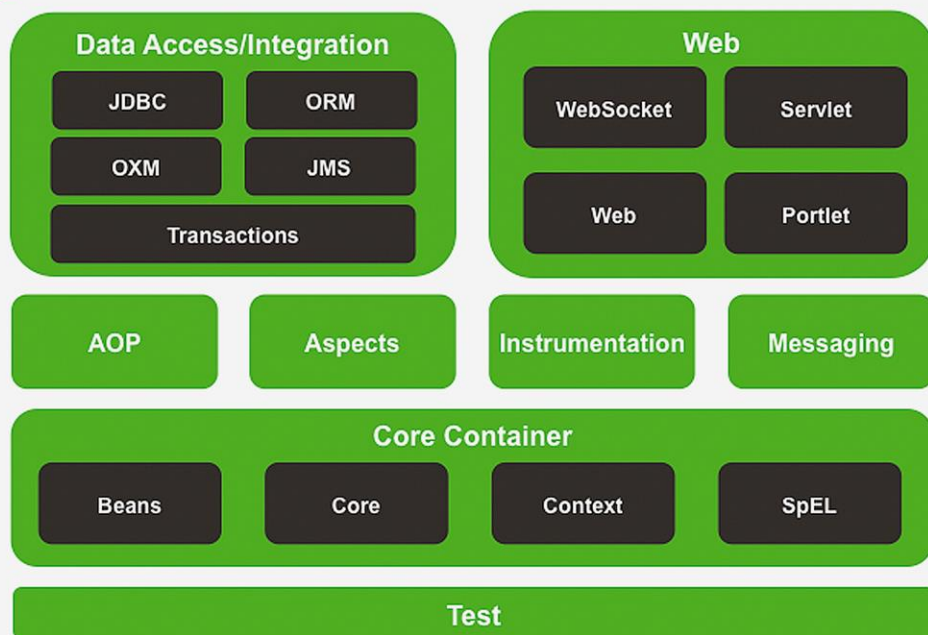
Жизненный цикл бинов



Spring Framework



Spring Framework Runtime



Spring Framework предоставляет около 20 модулей, которые можно использовать в зависимости от требований приложения.

Основной контейнер

Базовый контейнер состоит из модулей Core, Beans, Context и Expression Language, подробности которых следующие:

- **Core** обеспечивает основные части платформы, включая функции IoC и Dependency Injection.
- **Bean** предоставляет BeanFactory, которая представляет собой сложную реализацию фабричного шаблона.
- **Context** основан на прочной основе, предоставляемой модулями Core и Beans, и является средой для доступа к любым объектам, определенным и настроенным. Интерфейс ApplicationContext является координационным центром модуля Context.
- **SpEL** предоставляет мощный язык выражений для запросов и манипулирования графом объектов во время выполнения.

Доступ к данным/интеграция

- **JDBC** предоставляет уровень абстракции JDBC, который устраняет необходимость в утомительном кодировании, связанном с JDBC.
- **ORM** предоставляет слои интеграции для популярных API объектно-реляционного отображения, включая JPA, JDO, Hibernate и iBatis.
- **ОХМ** предоставляет уровень абстракции, который поддерживает реализации отображения объектов / XML для JAXB, Castor, XMLBeans, JiBX и XStream.
- **JMS** Java Messaging Service содержит функции для создания и потребления сообщений.
- **Transaction** поддерживает программное и декларативное управление транзакциями для классов, которые реализуют специальные интерфейсы, и для всех ваших POJO.

Web

- **Веб-** модуль обеспечивает базовые функции веб-интеграции, такие как функция многоэтапной загрузки файлов и инициализация контейнера IoC с использованием прослушивателей сервлетов и контекста веб-ориентированного приложения.
- **Web-MVC** содержит реализацию Spring-Model-View-Controller (MVC) для веб-приложений.
- **Web-Socket** обеспечивает поддержку двусторонней связи на основе WebSocket между клиентом и сервером в веб-приложениях.
- **Web-** портлета предоставляет реализацию MVC для использования в среде портлета и отражает функциональность модуля Web-Servlet.

Остальное

AOP обеспечивает реализацию аспектно-ориентированного программирования, позволяющую вам определять методы-перехватчики и указатели, чтобы четко отделить код, который реализует функциональность, которая должна быть отделена.

Модуль « **Аспекты** » обеспечивает интеграцию с AspectJ, который снова является мощной и зрелой средой AOP.

Instrumentation обеспечивает поддержку инструментария класса и реализации загрузчика классов для использования на определенных серверах приложений.

Модуль **обмена сообщениями** обеспечивает поддержку STOMP в качестве суб-протокола WebSocket для использования в приложениях. Он также поддерживает модель программирования аннотаций для маршрутизации и обработки сообщений STOMP от клиентов WebSocket.

Test поддерживает тестирование компонентов Spring с помощью каркасов JUnit или TestNG.

Паттерны в Spring

- Chain of Responsibility - это поведенческий паттерн проектирования, который позволяет передавать запросы последовательно по цепочке обработчиков. Каждый последующий обработчик решает, может ли он обработать запрос сам и стоит ли передавать запрос дальше по цепи. Ему Spring Security
- Singleton (Одиночка) - Паттерн Singleton гарантирует, что в памяти будет существовать только один экземпляр объекта, который будет предоставлять сервисы. Spring область видимости бина (scope) по умолчанию равна singleton и IoC-контейнер создаёт ровно один экземпляр объекта на Spring IoC-контейнер. Spring-контейнер будет хранить этот единственный экземпляр в кэше синглтон-бинов, и все последующие запросы и ссылки для этого бина получат кэшированный объект. Рекомендуется использовать область видимости singleton для бинов без состояния. Область видимости бина можно определить как singleton или как prototype (создаётся новый экземпляр при каждом запросе бина).
- Model View Controller (Модель-Представление-Контроллер) - Преимущество Spring MVC в том, что ваши контроллеры являются POJO, а не сервлетами. Это облегчает тестирование контроллеров. Стоит отметить, что от контроллеров требуется только вернуть логическое имя представления, а выбор представления остаётся за ViewResolver. Это облегчает повторное использование контроллеров при различных вариантах представления
- Front Controller (Контроллер запросов) - Spring предоставляет DispatcherServlet, чтобы гарантировать, что входящий запрос будет отправлен вашим контроллерам. Паттерн Front Controller используется для обеспечения централизованного механизма обработки запросов, так что все запросы обрабатываются одним обработчиком. Этот обработчик может выполнить аутентификацию, авторизацию, регистрацию или отслеживание запроса, а затем передать запрос соответствующему контроллеру. View Helper отделяет статическое содержимое в представлении, такое как JSP, от обработки бизнес-логики.

Паттерны в Spring

- Dependency injection и Inversion of control (IoC) (Внедрение зависимостей и инверсия управления) - IoC-контейнер в Spring, отвечает за создание объекта, связывание объектов вместе, конфигурирование объектов и обработку всего их жизненного цикла от создания до полного уничтожения. В контейнере Spring используется инъекция зависимостей (Dependency Injection, DI) для управления компонентами приложения. Эти компоненты называются "Spring-бины" (Spring Beans).
- Service Locator (Локатор служб) - ServiceLocatorFactoryBean сохраняет информацию обо всех бинах в контексте. Когда клиентский код запрашивает сервис (бин) по имени, он просто находит этот компонент в контексте и возвращает его. Клиентскому коду не нужно писать код, связанный со Spring, чтобы найти бин. Паттерн Service Locator используется, когда мы хотим найти различные сервисы, используя JNDI. Учитывая высокую стоимость поиска сервисов в JNDI, Service Locator использует кеширование. При запросе сервиса первый раз Service Locator ищет его в JNDI и кэширует объект. Дальнейший поиск этого же сервиса через Service Locator выполняется в кэше, что значительно улучшает производительность приложения.
- Observer-Observable (Наблюдатель) - Используется в механизме событий ApplicationContext. Определяет зависимость "один-ко-многим" между объектами, чтобы при изменении состояния одного объекта все его подписчики уведомлялись и обновлялись автоматически.
- Context Object (Контекстный объект) - Паттерн Context Object, инкапсулирует системные данные в объекте-контексте для совместного использования другими частями приложения без привязки приложения к конкретному протоколу. ApplicationContext является центральным интерфейсом в приложении Spring для предоставления информации о конфигурации приложения.
- Proxy (Заместитель) - позволяет подставлять вместо реальных объектов специальные объекты-заменители. Эти объекты перехватывают вызовы к оригинальному объекту, позволяя сделать что-то до или после передачи вызова оригиналу.
- Factory (Фабрика) - определяет общий интерфейс для создания объектов в суперклассе, позволяя подклассам изменять тип создаваемых объектов.
- Template (Шаблон) - Этот паттерн широко используется для работы с повторяющимся бойлерплейт кодом (таким как, закрытие соединений и т. п.).

АОП и составные части

Аспектно-ориентированное программирование (АОП) — это парадигма программирования, целью которой является повышение модульности за счет разделения междисциплинарных задач. Это достигается путем добавления дополнительного поведения к существующему коду без изменения самого кода.

Аспект (Aspect) - Это модуль, который имеет набор программных интерфейсов, которые обеспечивают сквозные требования. К примеру, модуль логирования будет вызывать АОП аспект для логирования. В зависимости от требований, приложение может иметь любое количество аспектов.

Объединённая точка (Join point) - Это такая точка в приложении, где мы можем подключить аспект. Другими словами, это место, где начинаются определённые действия модуля АОП в Spring.

Совет (Advice) - Это фактическое действие, которое должно быть предпринято до и/или после выполнения метода. Это конкретный код, который вызывается во время выполнения программы.

АОП и составные части

- before - Запускает совет перед выполнением метода.
- after - Запускает совет после выполнения метода, независимо от результата его работы (кроме случая остановки работы JVM).
- after-returning - Запускает совет после выполнения метода, только в случае его успешного выполнения.
- after-throwing - Запускает совет после выполнения метода, только в случае, когда этот метод “бросает” исключение.
- around - Запускает совет до и после выполнения метода. При этом инпоинты видят только начало и конец метода. Например, если метод выполняет транзакцию и где-то в середине кода try/catch поймал exception, транзакция все равно будет свершена, rollback не произойдет. В этом случае нужно пробрасывать ошибку за пределы метода.

Срез точек (Pointcut) - Срезом называется несколько объединённых точек (join points), в котором должен быть выполнен совет.

Введение (Introduction) - Это сущность, которая помогает нам добавлять новые атрибуты и/или методы в уже существующие классы.

Целевой объект (Target object) - Это объект на который направлены один или несколько аспектов.

Плетение (Weaving) - Это процесс связывания аспектов с другими объектами приложения для создания совета. Может быть вызван во время компиляции, загрузки или выполнения приложения.

Некоторые аннотации

- @Autowired - используется для автоматического связывания зависимостей в spring beans.
- @Bean - В классах конфигурации Spring, @Bean используется для для непосредственного создания бина.
- @Controller - класс фронт контроллера в проекте Spring MVC.
- @ConditionalOn* - Создает бин если выполняется условие. Condition - функциональный интерфейс, который содержит метод boolean matches(ConditionContext context, AnnotatedTypeMetadata metadata)
- @Scheduled - Таймер. Раз в сколько-то секунд обрабатывать.
- @Resource - Java аннотация, которой можно внедрить зависимость.
- @Required - применяется к методам-сеттерам и означает, что значение метода должно быть установлено в XML-файле. Если этого не будет сделано, то мы получим BeanInitializationException.
- @RequestMapping - используется для мапинга (связывания) с URL для всего класса или для конкретного метода обработчика

Некоторые аннотации

- `@ResponseBody` - позволяет отправлять Object в ответе. Обычно используется для отправки данных формата XML или JSON.
- `@ResponseBody` - используется для формирования ответа HTTP с пользовательскими параметрами (заголовки, http-код и т.д.). `ResponseBody` необходим, только если мы хотим кастомизировать ответ, добавив к нему статус ответа. Во всех остальных случаях будем использовать `@ResponseBody`.
- `@PathVariable` - задает динамический маппинг значений из URI внутри аргументов метода обработчика, т.е. позволяет вводить в URI переменную пути в качестве параметра
- `@Qualifier` - используется совместно с `@Autowired` для уточнения данных связывания, когда возможны коллизии (например одинаковых имен\типов).
- `@Service` - указывает что класс осуществляет сервисные функции.
- `@Scope` - указывает scope у spring bean.
- `@Configuration`, `@ComponentScan` и `@Bean` - для java based configurations.
- AspectJ аннотации для настройки aspects и advices, `@Aspect`, `@Before`, `@After`, `@Around`, `@Pointcut` и др.
- `@PageableDefault` - устанавливает значение по умолчанию для параметра разбиения на страницы

Различия @Component, @Service, @Repository, @Controller

Они все служат для обозначения класса как Бин.

- @Component - Spring определяет этот класс как кандидата для создания bean.
- @Service - класс содержит бизнес-логику и вызывает методы на уровне хранилища. Ничем не отличается от классов с @Component.
- @Repository - указывает, что класс выполняет роль хранилища (объект доступа к DAO). При этом отлавливает определенные исключения персистентности и пробрасывает их как одно непроверенное исключение Spring Framework. Для этого Spring оборачивает эти классы в прокси, и в контекст должен быть добавлен класс PersistenceExceptionTranslationPostProcessor
- @Controller - указывает, что класс выполняет роль контроллера MVC. Диспетчер сервлетов просматривает такие классы для поиска @RequestMapping.

@Inject and @Resource

Размещается над полями, методами, и конструкторами с аргументами. @Inject как и @Autowired в первую очередь пытается подключить зависимость по типу, затем по описанию и только потом по имени. Это означает, что даже если имя переменной ссылки на класс отличается от имени компонента, но они одинакового типа, зависимость все равно будет разрешена

@Resource (java) пытается получить зависимость: по имени, по типу, затем по описанию. Имя извлекается из имени аннотируемого сеттера или поля, либо берется из параметра name.

@Inject (java) или @Autowired (spring) в первую очередь пытается подключить зависимость по типу, затем по описанию и только потом по имени.